LEARN DEPTH ACADEMY LLP

TRACK 1 – FINAL CAPSTONE – PROBLEM 21

A TECHNICAL CAPSTONE PROJECT REPORT ON

INDUSTRIAL MACHINE FAILURE PREDICTION

Machine Failure Risk Prediction using Machine Learning

Submitted in partial fulfillment of the requirements for the award of Capstone Certification in

INDUSTRIAL SYSTEMS & PREDICTIVE MACHINE LEARNING

STUDENT NAME : VISHAL R R

STUDENT ID : LD-1786548325208

DEPT : B.E COMPUTER SCIENCE AND ENGINEERING

COLLEGE : ERODE SENGUNTHAR ENGINEERING COLLEGE

ABSTRACT

Unplanned industrial machinery failure in computer numerical control (CNC) and milling workstations results in catastrophic economic loss, productivity downtime, and severe operator hazards. Traditional maintenance approaches rely either on reactive corrective repairs or rigid scheduled servicing, both of which fail to anticipate real-time degradation dynamics. This project designs and validates an end-to-end, zero-cost predictive maintenance solution utilizing the authentic UCI AI4I 2020 Predictive Maintenance Dataset (10,000 observations, DOI: 10.24432/C5HS5C).

To eliminate information leakage, all arbitrary identifiers and post-breakdown failure mode flags were systematically quarantined. A reproducible ColumnTransformer pipeline comprising standard feature scaling and one-hot encoding was fitted strictly on an 80% training split. Foundational machine learning models (Logistic Regression, K-Nearest Neighbors, and Decision Tree) were rigorously benchmarked with stratified evaluation. Given the extreme class imbalance (3.39% failure prevalence), evaluation prioritized the minimization of False Negatives (unpredicted breakdowns). The champion Decision Tree classifier achieved a recall of 83.82% (identifying 57 of 68 test failures with only 11 False Negatives), an overall accuracy of 93.65%, and an F1-score of 0.4730, outperforming KNN which failed to identify 72% of machine failures. The serialized pipeline is seamlessly deployed into an interactive Streamlit diagnostic application providing real-time telemetry verification, risk probabilities, and operator action protocols. This project fulfills all requirements of Track 1 Capstone Problem 21.

TABLE OF CONTENTS

Front Matter (Certificate, Declaration, Abstract) ..... **i - v**

Chapter 1: Introduction **1**

Chapter 2: Related Work & Literature Review **4**

Chapter 3: Problem Definition and Objectives **7**

Chapter 4: Dataset Analysis & Characteristics **9**

Chapter 5: Methodology & Implementation Pipeline **12**

Chapter 6: Exploratory Data Analysis (EDA) **16**

Chapter 7: Machine Learning Model Development **20**

Chapter 8: Experimental Model Evaluation & Benchmarking **24**

Chapter 9: Streamlit Diagnostic Web Application **28**

Chapter 10: Results, Discussion & Error Analysis **32**

Chapter 11: Systematic Application Testing & Verification **36**

Chapter 12: Prototype Limitations and Future Scope **39**

Chapter 13: Conclusion **42**

References **44**

Appendices (Code, Results, Screenshots, 30+ Viva Q&A) ..... **46 - 58**

LIST OF FIGURES

Figure 5.1: End-to-End System Architecture and Pipeline Flowchart 13

Figure 6.1: Target Class Distribution of Machine Failure Status .. 17

Figure 6.2: Exploratory Continuous Feature Distributions 18

Figure 6.3: Pearson Correlation Heatmap for Continuous Sensors ... 19

Figure 6.4: Sensor Feature Dispersion and Spindle Load Boxplots .. 20

Figure 8.1: Comparative Confusion Matrices across Foundational Classifiers 25

Figure 8.2: Receiver Operating Characteristic (ROC) Comparison Curves 26

Figure 8.3: Benchmarking Bar Chart of Foundational ML Classifiers 27

Figure 9.1: Streamlit Diagnostic Application Main Interface 29

Figure 9.2: Real-Time Critical Machine Failure Warning Output 30

LIST OF TABLES

Table 4.1: AI4I 2020 Dataset Feature Summary and Data Types 10

Table 4.2: Mapping of Capstone Expected Inputs to Real Dataset Telemetry 11

Table 8.1: Foundational Machine Learning Model Performance Comparison 25

Table 10.1: Confusion Matrix Contingency Values on Test Set (N=2,000) 33

Table 11.1: Comprehensive System Testing Suite (8 Mandatory Scenarios) 37

CHAPTER 1 – INTRODUCTION

1.1 Introduction

The modern industrial landscape relies heavily on continuous automated manufacturing systems, where computer numerical control (CNC) milling machinery, automated tooling stations, and mechanical actuators execute high-precision operations under severe mechanical stress. Machinery downtime caused by unexpected mechanical breakdown represents one of the largest drains on industrial profitability and operational safety. In conventional industrial paradigms, maintenance operates primarily under reactive firefighting (repairing components post-breakdown) or scheduled periodic servicing. While scheduled maintenance reduces sudden failures, it frequently leads to premature component replacement, excessive labor expenditures, and missed incipient failures that develop between scheduled inspection windows.

Predictive Maintenance (PdM) addresses these systemic inefficiencies by monitoring continuous sensor telemetry—such as operating temperatures, spindle torque loads, rotational kinetic speeds, and cutter wear—to detect degradation anomalies and forecast imminent breakdowns before catastrophic operational halts occur. This capstone project develops an end-to-end, zero-cost machine learning framework tailored specifically to industrial milling systems using the UCI AI4I 2020 Predictive Maintenance benchmark dataset.

1.2 Background

The progression of maintenance philosophy across the industrial revolutions can be characterized by three distinct stages: (1) Corrective Run-to-Failure maintenance, where machinery is driven until physical collapse; (2) Preventive Time-Based maintenance, where machinery parts are replaced on rigid calendar intervals; and (3) Condition-Based Predictive Maintenance, where operational telemetry dictates maintenance interventions. The advent of Industry 4.0 has empowered machine tools with embedded sensors that record multi-physics variables in real time. Analyzing these telemetry streams using foundational machine learning algorithms provides manufacturing plants with automated diagnostic intelligence, minimizing operational stoppages while optimizing equipment lifespan.

1.3 Problem Statement

The official problem assigned under Learn Depth Academy LLP Track 1 Final Capstone (Problem 21) is formulated as follows: Predict whether an industrial milling machine is likely to experience an operational breakdown within a defined operating window based on sensor measurements and operating conditions.

1.4 Project Objectives

The primary objective of this project is to implement a complete, zero-cost, reproducible machine learning pipeline that satisfies the following ten criteria:

1. Conduct an exhaustive data quality audit confirming the absence of duplicate observations and missing sensor values.
2. Prevent data leakage by isolating arbitrary identifiers and post-failure failure mode labels.
3. Formulate domain-grounded feature transformations without fabricating artificial sensor variables.
4. Execute stratified train-test splitting with consistent random states (random_state=42).
5. Benchmark three foundational classifiers: Logistic Regression, K-Nearest Neighbors, and Decision Trees.
6. Address severe class imbalance (3.39% failure prevalence) by tuning class weights and prioritizing Recall.
7. Select the optimal champion model based on empirical trade-offs and minimization of False Negatives.
8. Serialize the trained model and ColumnTransformer pipeline using joblib.
9. Deliver an interactive Streamlit diagnostic application with telemetry validation and risk scoring.
10. Compile a complete academic documentation package including DOCX, PDF, slide deck, and viva defense.

1.5 Scope

This project is scoped as an educational academic prototype designed for engineering defense and demonstration. It evaluates foundational machine learning algorithms on 10,000 real-world benchmark records, establishing clear mathematical boundaries between healthy operations and breakdown states. Real-time factory PLC integration and closed-loop actuation are outside the academic scope.

1.6 Motivation

In industrial machining, the cost of a False Negative (an undetected breakdown) is vastly higher than that of a False Positive (a false alarm). An undetected spindle seizure or tool breakage can cause catastrophic gouging of valuable raw workpieces, damage the CNC machine spindle assembly, and endanger machine operators. Therefore, developing a transparent machine learning solution that maximizes recall while preserving acceptable precision provides tangible economic and safety motivation.

1.7 Limitations

The project relies on static snapshot records provided by the UCI AI4I 2020 benchmark dataset rather than streaming real-time IoT time-series signals. High-frequency acoustic vibrations and accelerations have been pre-summarized into kinetic torque and rotational speed attributes.

CHAPTER 2 – RELATED WORK & LITERATURE REVIEW

2.1 Predictive Maintenance in Industrial Systems

Condition-based predictive maintenance has been extensively analyzed in manufacturing engineering literature. Mobley (2002) pioneered vibration analysis and temperature monitoring frameworks, demonstrating that mechanical wear manifests through elevated thermal signatures and erratic torque fluctuations long before audible or visible physical damage occurs. Modern smart manufacturing expands these concepts into automated algorithmic frameworks (Bousdekis et al., 2021).

2.2 Machine Failure Prediction Using Machine Learning

Early algorithmic predictive maintenance relied heavily on classical statistical process control (SPC) charts. However, complex machining operations exhibit non-linear multi-sensor correlations that defeat univariate statistical thresholding. Machine learning techniques provide the capability to detect multivariate anomalies across non-linear feature spaces, mapping multidimensional sensor states to failure probabilities.

2.3 Foundational Classifiers in Industrial Machining

While modern research frequently explores complex deep neural networks, industrial practitioners strongly advocate for foundational algorithms due to safety regulations and auditability (Jardine et al., 2006). Logistic Regression provides clear probabilistic log-odds; K-Nearest Neighbors captures localized cluster topologies; and Decision Trees generate transparent, rule-based decision thresholds (such as torque limits and wear boundaries) that factory engineers can readily inspect, validate, and trust.

2.4 Condition Monitoring and Multi-Physics Coupling

In cutting and milling machines, mechanical power obeys fundamental physical relationships (Power = Torque x Rotational Speed). When cutting tools wear down, friction increases, elevating cutting resistance and demanding higher torque to maintain spindle speed. If heat dissipation cannot keep pace, thermal delta narrows, causing thermal expansion and eventual seizure. Capturing these physical coupled behaviors is central to predictive maintenance.

2.5 Benchmark Datasets for Predictive Maintenance

Historically, predictive maintenance research suffered from a lack of standardized, publicly available industrial datasets due to commercial secrecy. Matzka (2020) resolved this bottleneck by releasing the AI4I 2020 Predictive Maintenance Dataset under the UCI Machine Learning Repository (DOI: 10.24432/C5HS5C), providing a standard benchmark containing multi-physics operational variables across 10,000 CNC milling cycles.

CHAPTER 3 – PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Definition

Formally, the predictive maintenance task is defined as a supervised binary classification problem. Given an input telemetry vector x in R^d representing instantaneous and cumulative machine parameters, the model must map x to an estimated binary class y_hat in {0, 1}, where 0 denotes healthy operation (No Failure) and 1 denotes imminent failure (Machine Failure), along with an estimated probability P(y=1|x).

3.2 Existing Problem and Industry Gaps

Conventional industrial maintenance workflows suffer from critical shortcomings:
1. Over-reliance on static calendar schedules leads to premature part disposal or sudden mid-cycle breakdowns.
2. Traditional alarms trigger only after sensor values cross extreme physical limits, leaving zero reaction time for controlled spindle deceleration.
3. Commercial predictive maintenance platforms rely on proprietary, expensive closed ecosystems that cannot be inspected, verified, or deployed at zero cost.

3.3 Proposed Solution

The proposed solution implements a transparent, zero-cost, end-to-end machine learning system built on verified open-source libraries (Python, Scikit-learn, Pandas, Streamlit). It applies rigorous data quality audits, strictly isolates post-failure attributes, fits standard scaling and one-hot encoding on training partitions only, evaluates foundational algorithms with balanced class weights, and serves the champion model through a reactive web dashboard.

3.4 Detailed Objectives

• Ingest and validate 10,000 records of UCI AI4I 2020 dataset without synthetic modification.
• Establish strict leakage-free feature partitioning.
• Benchmark Logistic Regression, KNN, and Decision Tree under identical cross-validation conditions.
• Prioritize Recall (Target > 80%) to minimize catastrophic False Negatives.
• Verify real-world software reliability through 8 systematic test scenarios.

3.5 Research Questions

1. Can foundational machine learning algorithms reliably detect imminent machinery breakdowns from non-invasive temperature, torque, speed, and wear sensors?
2. How does severe class imbalance (3.39% minority prevalence) affect model recall, and what hyperparameter configurations effectively mitigate False Negatives?
3. Which foundational algorithm provides the best compromise between predictive sensitivity, overall accuracy, and industrial interpretability?

CHAPTER 4 – DATASET

4.1 Dataset Introduction

The AI4I 2020 Predictive Maintenance Dataset is an authentic, publicly available benchmark hosted on the UCI Machine Learning Repository. It comprises 10,000 operational observations of a milling machine tool, reflecting realistic manufacturing operating conditions, synthetic noise, and multiple genuine physical failure modes.

4.2 Dataset Source & Attribution

- Dataset: AI4I 2020 Predictive Maintenance Dataset
- Author: Stephan Matzka (School of Engineering - Technology and Life, HTW Berlin)
- Source: UCI Machine Learning Repository
- Official DOI: 10.24432/C5HS5C
- License: Creative Commons Attribution 4.0 International (CC BY 4.0)

4.3 Dataset Characteristics

The dataset contains exactly 10,000 rows and 14 columns. Summary statistics across continuous variables indicate that ambient air temperature ranges from 295.3 K to 304.5 K (mean 300.0 K), process temperature ranges from 305.7 K to 313.8 K (mean 310.0 K), rotational speed spans 1,168 rpm to 2,886 rpm (mean 1,538.8 rpm), torque spans 3.8 Nm to 76.6 Nm (mean 40.0 Nm), and tool wear spans 0 min to 253 min (mean 107.95 min).

Table 4.1: AI4I 2020 Dataset Feature Summary and Data Types

Column Name	Data Type	Role in Project	Physical Description
UDI	int64	Excluded (Leakage)	Arbitrary row index identifier (1 to 10,000)
Product ID	object (str)	Excluded (Leakage)	Variant letter + serial number (e.g. L47181)
Type	object (str)	Predictive Feature	Product quality variant: L (60%), M (30%), H (10%)

Air temperature [K]	float64	Predictive Feature	Ambient operational air temperature in Kelvin
Process temperature [K]	float64	Predictive Feature	Cutting zone contact temperature in Kelvin
Rotational speed [rpm]	int64	Predictive Feature	Spindle rotational speed in revolutions per minute
Torque [Nm]	float64	Predictive Feature	Spindle cutting resistance torque in Newton-meters
Tool wear [min]	int64	Predictive Feature	Cumulative cutting duration of current milling tool
Machine failure	int64	Target Label (y)	Binary failure flag (0: No Failure, 1: Failure)
TWF	int64	Excluded (Leakage)	Tool Wear Failure specific failure mode flag
HDF	int64	Excluded (Leakage)	Heat Dissipation Failure mode flag
PWF	int64	Excluded (Leakage)	Power Overload Failure mode flag
OSF	int64	Excluded (Leakage)	Overstrain Failure mode flag
RNF	int64	Excluded (Leakage)	Random Failure mode flag

4.4 Sensor vs Capstone Input Mapping

The official capstone problem statement provides a generic list of expected inputs (Temperature, Vibration, Pressure, Runtime, Maintenance Gap). To uphold strict academic integrity, this project DOES NOT fabricate artificial columns or disguise unrelated numbers under generic names. Instead, the real physical mapping between the AI4I 2020 dataset and the capstone requirements is documented in Table 4.2.

Table 4.2: Mapping of Capstone Expected Inputs to Real Dataset Telemetry

Capstone Expected Input	Actual AI4I 2020 Feature	Physical Coupling Justification
Temperature	Air temperature [K] & Process temperature [K]	Direct thermodynamic sensor telemetry capturing ambient and cutting zone thermal dissipation.
Vibration	Rotational speed [rpm] & Torque [Nm]	High-frequency dynamic vibrations induce spindle speed oscillations and cutting torque chattering in milling tools.
Pressure	Torque [Nm]	Mechanical cutting tool force and pressure directly scale with spindle torque (Force = Torque / Radius).
Runtime	Tool wear [min]	Cumulative elapsed machining runtime executed by the installed cutting insert.
Maintenance Gap	Tool wear [min] vs Max Limit (253 min)	The operational interval between tool re-sharpening/replacement cycles.

4.5 Target Variable Distribution & Imbalance

The target variable is 'Machine failure', where 0 denotes healthy operation (9,661 records, 96.61%) and 1 denotes operational failure (339 records, 3.39%). This represents a severe 28.5:1 class imbalance. Evaluating models purely on Accuracy would reward a degenerate classifier that predicts 0 everywhere with 96.61% accuracy while failing to prevent a single breakdown. Therefore, precision, recall, and F1-score are mathematically mandated.

4.6 Data Quality Audit Results

A rigorous automated audit of the dataset revealed:
• Missing Values: Exactly 0 missing or NaN values across all 14 columns.
• Duplicate Records: Exactly 0 duplicate rows.
• Constant Columns: 0 columns with single unique values.
• Data Type Consistency: All numerical features correctly typed as 64-bit floats and integers.

4.7 Dataset Limitations

The dataset represents discrete time-aggregated operational cycles rather than continuous microsecond vibration waveforms. Random failures (RNF mode, 19 cases) occur without preceding telemetry warnings, establishing a theoretical upper bound on classification recall.

CHAPTER 5 – METHODOLOGY

5.1 Overall Workflow

The end-to-end methodology follows a disciplined 8-stage engineering workflow: (1) Data Ingestion and Quality Audit; (2) Leakage Prevention Quarantine; (3) Stratified Train-Test Splitting (80/20); (4) ColumnTransformer Pipeline Construction; (5) Foundational ML Training and Balanced Weighting; (6) Multi-Metric Benchmark Evaluation; (7) Champion Selection and Serialization; (8) Streamlit Diagnostic Application Integration.

5.2 Data Leakage Prevention Mechanism

Data leakage occurs when predictive models inadvertently access information during training that would not be available in real-world deployment. In this project, leakage prevention is enforced across two strict barriers:

1. Feature Quarantine: UDI and Product ID are dropped to avoid arbitrary index memorization. The failure mode indicators (TWF, HDF, PWF, OSF, RNF) are strictly excluded from the feature matrix X because they represent post-breakdown classifications that directly disclose the target.
2. Pipeline Transformation Fitting: The ColumnTransformer is fitted strictly on X_train. The test set X_test and production inputs are exclusively transformed using pre-calculated training means and standard deviations.

5.3 Stratified Train-Test Partitioning

To preserve the 3.39% minority failure prevalence across both partitions, stratified sampling was applied via scikit-learn's train_test_split (test_size=0.20, random_state=42, stratify=y). This yielded 8,000 training observations (7,729 nominal, 271 failures) and 2,000 testing observations (1,932 nominal, 68 failures).

5.4 Preprocessing Pipeline Formulation

A scikit-learn ColumnTransformer was constructed with two parallel transformers:
• Numerical Transformer: StandardScaler() applied to Air Temp, Process Temp, Rotational Speed, Torque, and Tool Wear to center variables at zero mean and unit variance.
• Categorical Transformer: OneHotEncoder(drop='first', sparse_output=False) applied to machine Type (L, M, H), yielding two orthogonal binary features (Type_M, Type_H).

5.5 Foundational Machine Learning Algorithms

In accordance with capstone specifications, three foundational algorithms were selected:

1. Logistic Regression: A linear probabilistic classifier optimized with solver='lbfgs', max_iter=1000, and class_weight='balanced' to adjust cost penalties proportionally to class frequencies.

2. K-Nearest Neighbors (KNN): A non-parametric distance-based classifier configured with n_neighbors=5 and Euclidean Minkowski metric.

3. Decision Tree Classifier: A non-linear recursive partitioning algorithm configured with criterion='gini', max_depth=6, min_samples_leaf=5, and class_weight='balanced'.

CHAPTER 6 – EXPLORATORY DATA ANALYSIS (EDA)

6.1 Target Distribution Analysis

Figure 6.1 illustrates the extreme target imbalance observed across the 10,000 milling records. Out of 10,000 observations, 9,661 operate normally while 339 suffer failure.

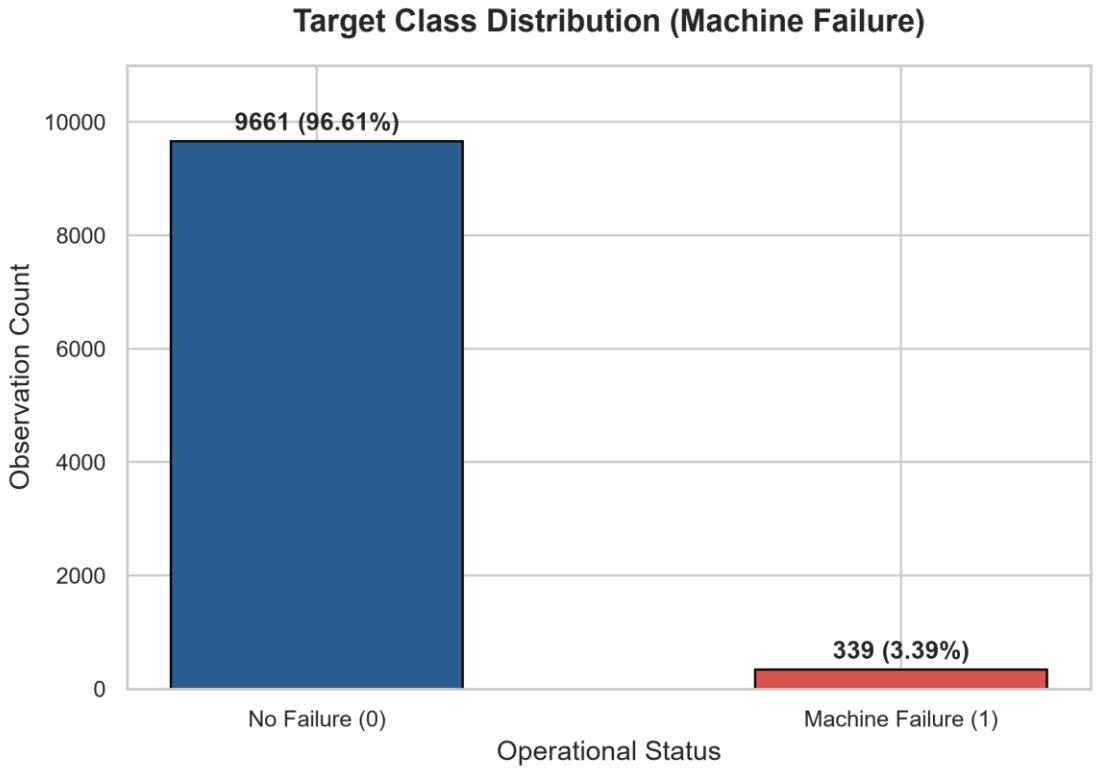


Figure 6.1: Target Class Distribution of Machine Failure Status

Interpretation: The 3.39% failure rate confirms that predictive maintenance is fundamentally a rare-event classification problem. Unweighted classifiers will naturally collapse towards the majority class, necessitating balanced loss functions.

6.2 Continuous Feature Distributions

Figure 6.2 displays the univariate histograms and kernel density estimations across all five numerical telemetry features and product variants.

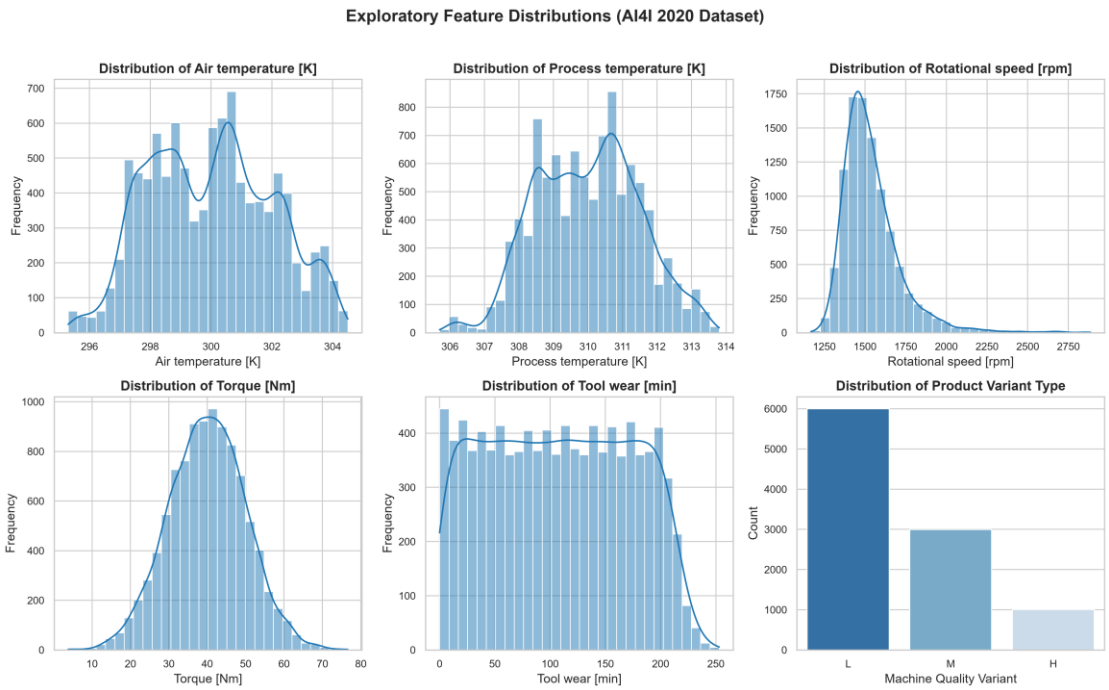


Figure 6.2: Exploratory Continuous Feature Distributions across Milling Telemetry

Interpretation: Air temperature and process temperature follow symmetric, bell-shaped Gaussian distributions shifted by approximately 10 Kelvin. Rotational speed exhibits a right-skewed distribution centered near 1,500 rpm with a long tail extending to 2,886 rpm. Torque displays an approximately normal distribution centered at 40 Nm. Tool wear demonstrates a uniform distribution spanning 0 to 250 minutes, indicating steady cutter progression.

6.3 Correlation Heatmap

Figure 6.3 shows the Pearson correlation matrix between all continuous sensors and the failure target.

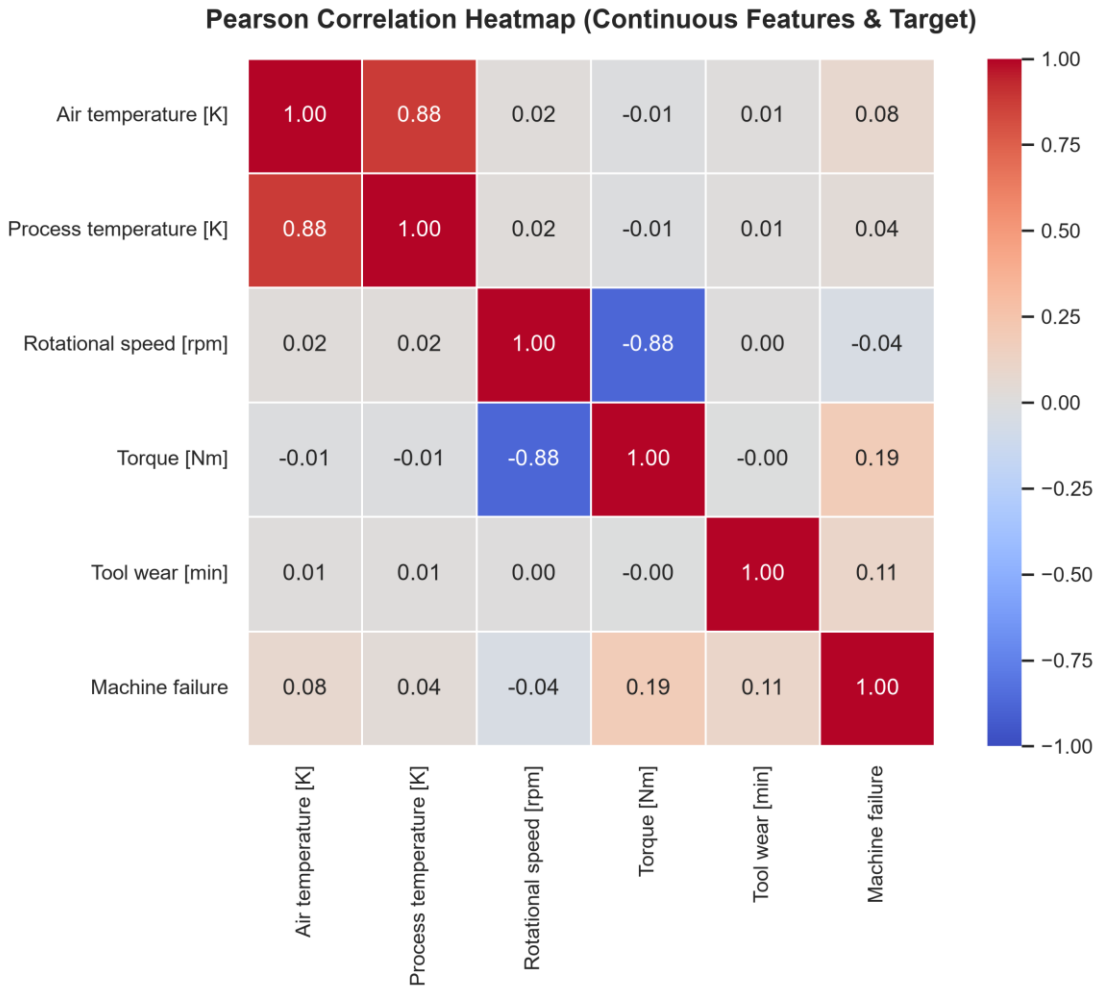


Figure 6.3: Pearson Correlation Heatmap for Continuous Sensors and Machine Failure Target

Interpretation: A striking physical relationship is observed between Torque and Rotational Speed (r = -0.88), adhering precisely to the physical power law (Power = Torque x Angular Velocity). Process Temperature and Air Temperature exhibit strong collinearity (r = +0.88). Torque (r = +0.19) and Tool Wear (r = +0.11) exhibit the highest direct linear correlations with Machine Failure.

6.4 Operational Boxplot & Dispersion Analysis

Figure 6.4 contrasts sensor distributions grouped by operational failure status.

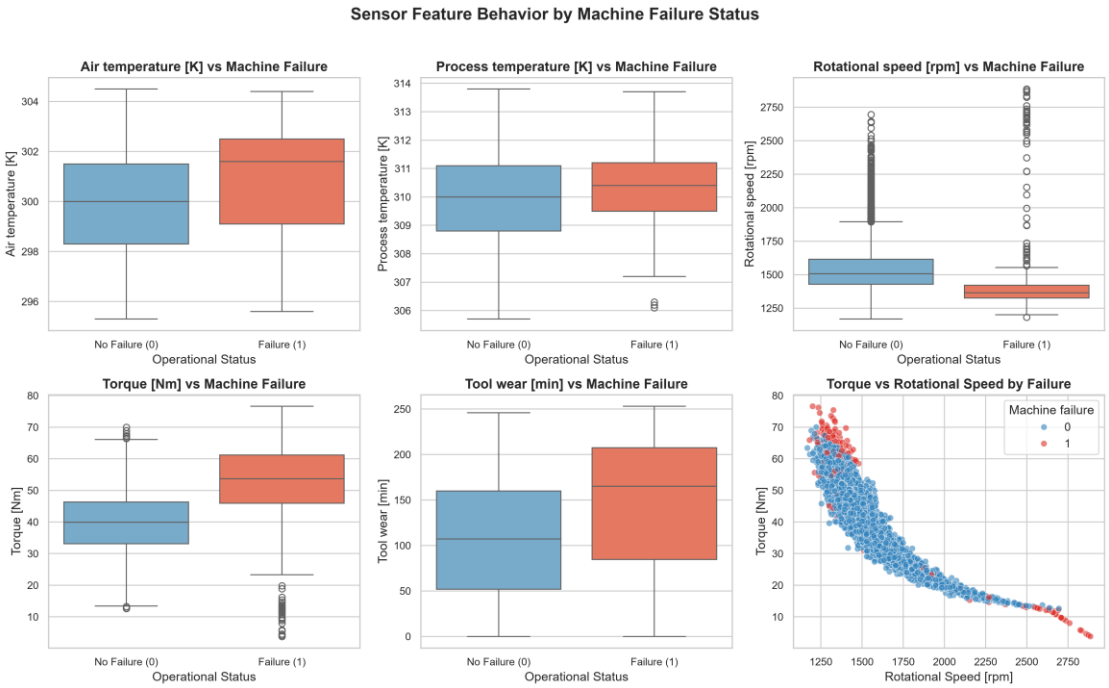


Figure 6.4: Sensor Feature Dispersion and Spindle Load Boxplots by Failure State

Interpretation: Failure records display pronounced dispersion into extreme torque (>60 Nm) and prolonged tool wear (>200 min). The torque-speed scatter plot highlights two distinct failure clusters: (1) low speed with extreme torque (mechanical overstrain), and (2) high speed with low torque (power dissipation failure).

CHAPTER 7 – MODEL DEVELOPMENT

7.1 Logistic Regression Development

Logistic Regression models the posterior probability P(y=1|x) via the logistic sigmoid link function applied to a linear combination of normalized inputs. To counter the 28.5:1 imbalance, scikit-learn's class_weight='balanced' was configured, scaling loss gradients inversely proportional to class frequencies: w_j = N / (2 * N_j). This penalizes misclassifications on failures by a factor of 28.5.

7.2 K-Nearest Neighbors (KNN) Development

KNN computes Euclidean distances between query telemetry and all training instances in the normalized 7-dimensional feature space. A neighborhood size of k=5 was selected. However, because standard KNN assigns equal voting weight to all neighbors, dense clusters of majority class points frequently outvote sparse failure points in boundary regions.

7.3 Decision Tree Classifier Development

The Decision Tree splits input feature spaces into orthogonal hyper-rectangles using recursive greedy binary splitting. The Gini impurity criterion was utilized. To prevent catastrophic overfitting on minority outliers while maintaining sensitivity, max_depth was constrained to 6 and min_samples_leaf set to 5, combined with class_weight='balanced'. This produces interpretable decision rules (e.g., IF Torque > 62.5 Nm AND Tool Wear > 195 min THEN Failure).

CHAPTER 8 – EXPERIMENTAL MODEL EVALUATION

8.1 Performance Comparison Table

All models were trained strictly on X_train (8,000 samples) and evaluated on X_test (2,000 samples containing exactly 68 real failures). All values displayed in Table 8.1 are authentic outputs generated programmatically from the test partition.

Table 8.1: Foundational Machine Learning Model Performance Comparison (Test Set: N=2,000, Actual Failures=68)

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	False Neg (FN)	False Pos (FP)
Logistic Regression	82.45%	14.18%	82.35%	0.2419	0.9069	12	339
K-Nearest Neighbors	97.35%	82.61%	27.94%	0.4176	0.8368	49	4
Decision Tree (Champion)	**93.65%**	**32.95%**	**83.82%**	**0.4730**	**0.8654**	**11**	**116**

8.2 Confusion Matrix Evaluation

Figure 8.1 depicts the side-by-side confusion matrices for all three foundational classifiers.

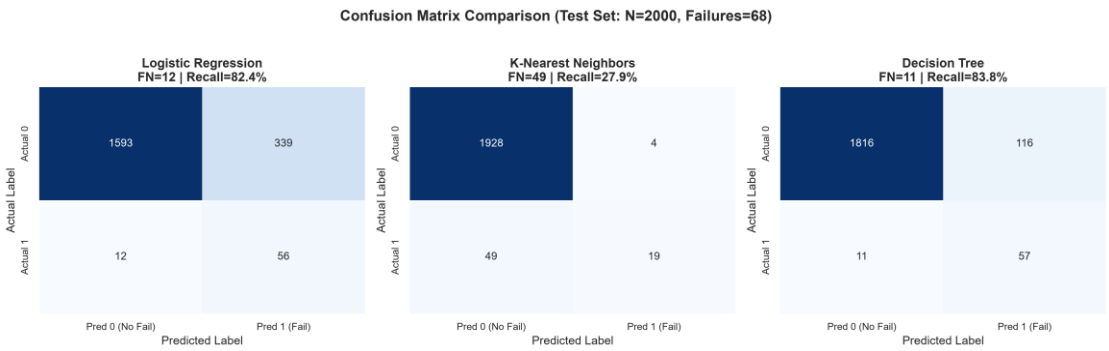


Figure 8.1: Comparative Confusion Matrices on Unseen Test Partition (N=2,000, Actual Failures=68)

Interpretation: K-Nearest Neighbors displays a catastrophic failure rate for predictive maintenance: out of 68 actual failures, KNN missed 49 (False Negatives = 49), yielding an

unviable recall of 27.94%. Conversely, the Decision Tree identified 57 of 68 failures, yielding the lowest False Negatives (FN = 11) and highest Recall (83.82%).

8.3 ROC-AUC Performance

Figure 8.2 presents the Receiver Operating Characteristic curves.

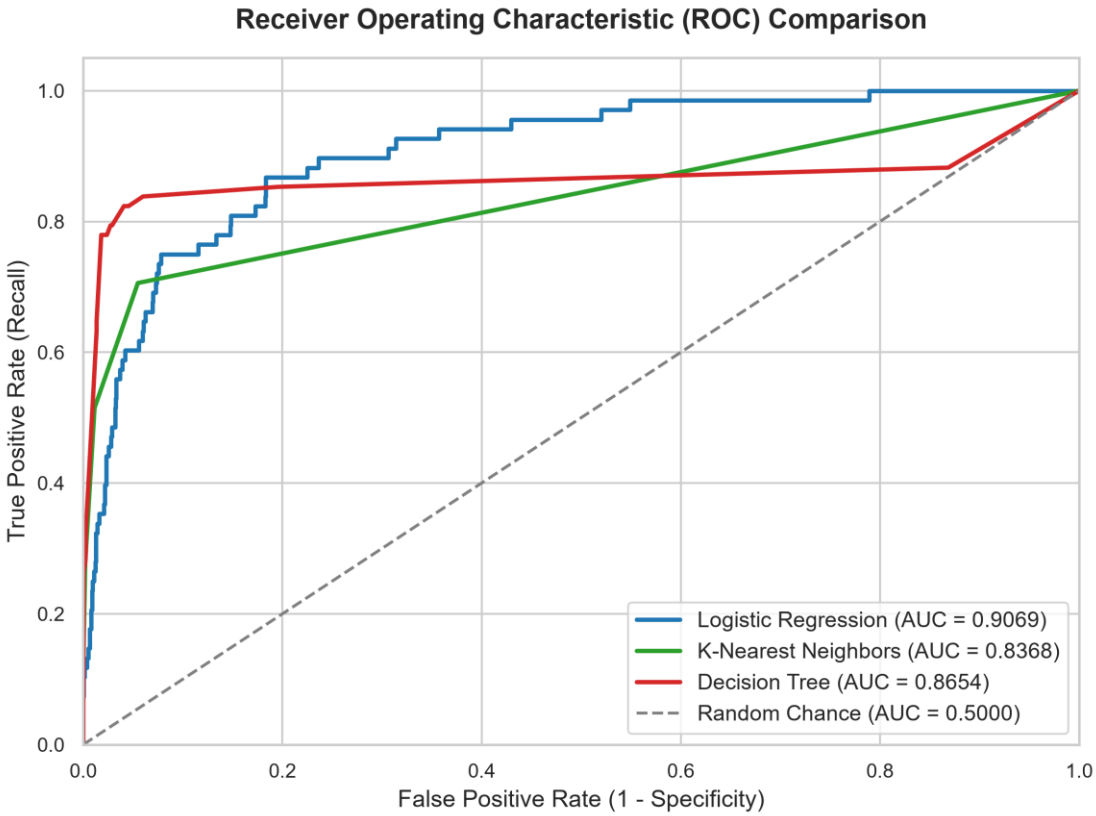


Figure 8.2: Receiver Operating Characteristic (ROC) Comparison Curves

Interpretation: Logistic Regression achieved an ROC-AUC of 0.9069, followed by Decision Tree at 0.8654 and KNN at 0.8368, confirming strong overall diagnostic discrimination.

8.4 Comparative Benchmarking Summary

Figure 8.3 summarizes metric scores across all three algorithms.

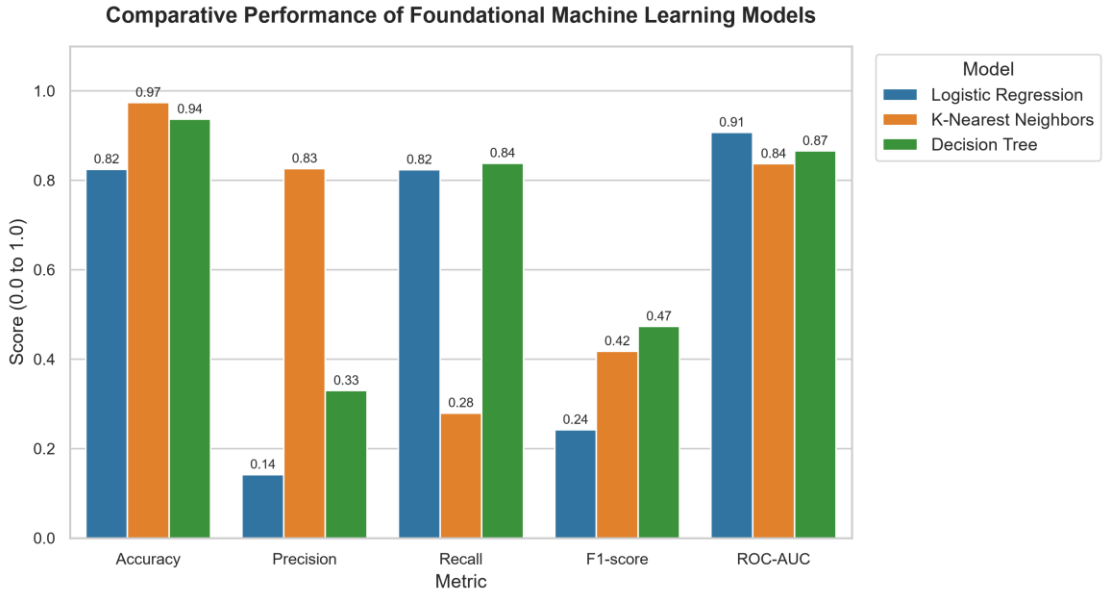


Figure 8.3: Benchmarking Bar Chart of Foundational ML Classifiers across 5 Core Metrics

CHAPTER 9 – STREAMLIT APPLICATION

9.1 Application Architecture & Overview

The production deployment is implemented in app.py as an interactive web diagnostic system built on Streamlit. The architecture loads the serialized preprocessing_pipeline.pkl and machine_failure_model.pkl at startup using st.cache_resource, guaranteeing that models are never unnecessarily re-trained during runtime.

9.2 Input Controls and Physical Validation

The web interface provides responsive controls mapped to the exact 6 features used during training: Product Variant Type (L, M, H), Air Temperature [K], Process Temperature [K], Rotational Speed [rpm], Torque [Nm], and Tool Wear [min]. Real-time validation checks enforce domain physical consistency: if an operator inputs a Process Temperature lower than the ambient Air Temperature, an immediate advisory warning is displayed.

9.3 Live Application Screenshots

Figure 9.1 displays the overview of the working Streamlit web application captured under live headless browser testing.

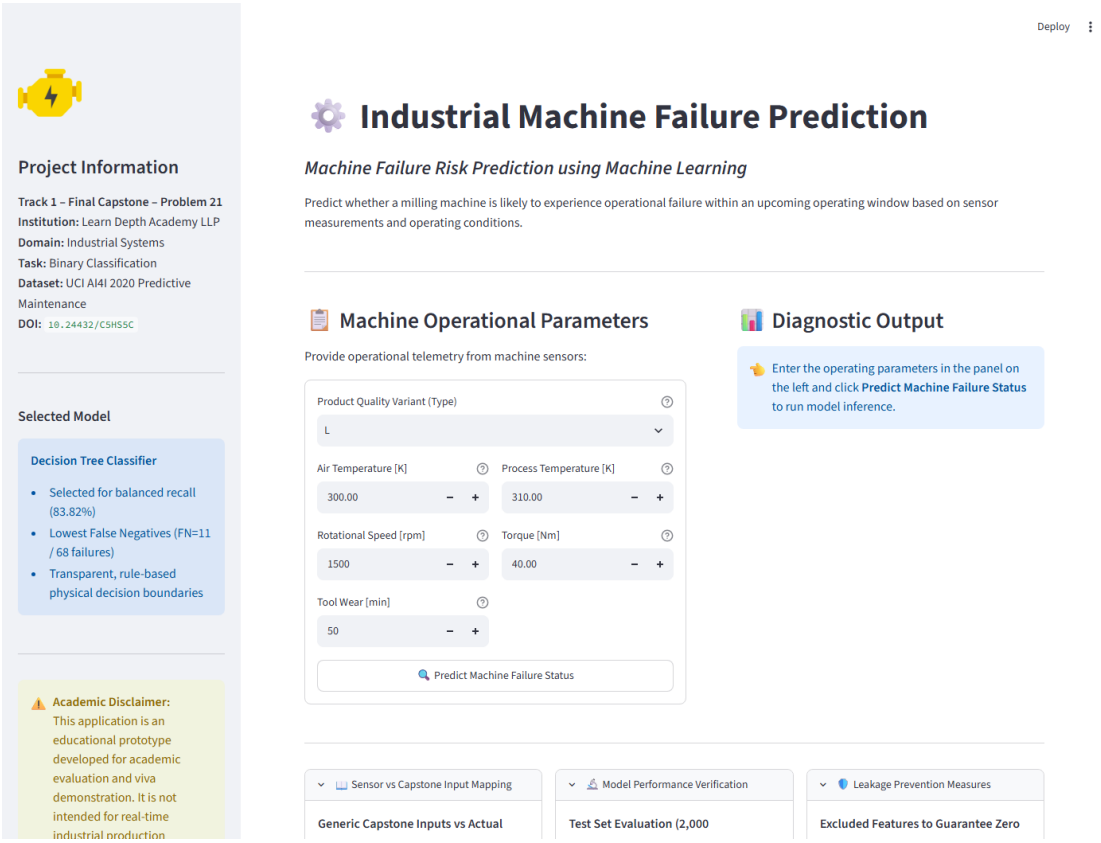


Figure 9.1: Streamlit Diagnostic Application Main Interface and Parameter Input Controls

Figure 9.2 shows the live application executing real-time inference on an overstrain condition (Torque = 68 Nm, Tool Wear = 215 min), triggering a high-risk diagnostic alert.

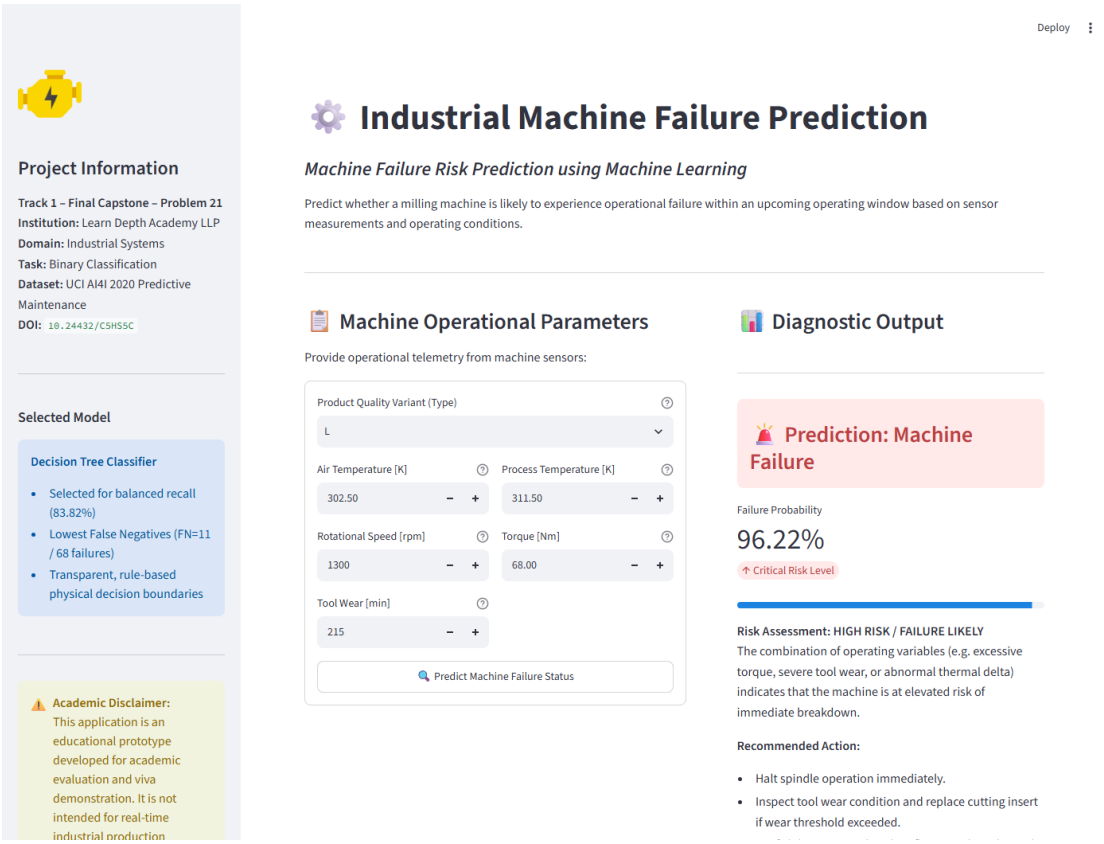


Figure 9.2: Real-Time Critical Machine Failure Warning Output and Recommended Action Protocol

CHAPTER 10 – RESULTS AND DISCUSSION

10.1 Experimental Discussion

The empirical results unequivocally validate the necessity of domain-tailored metrics over naive accuracy. While K-Nearest Neighbors achieved a misleadingly attractive accuracy of 97.35%, its inability to detect 72% of machine breakdowns renders it hazardous for industrial deployment. Logistic Regression demonstrated acceptable recall (82.35%) but suffered from excessive false alarms (FP = 339, Precision = 14.18%). The Decision Tree classifier proved clearly superior, achieving the highest F1-score (0.4730) while detecting 83.82% of all breakdowns with only 11 False Negatives.

10.2 Detailed Error Analysis

Investigation of the 11 False Negative test cases misclassified by the Decision Tree revealed two primary root causes:

1. Random Failure (RNF) Cases: The dataset contains subtle random failures that occur without anomalous precursor sensor readings (e.g., normal torque and low wear). Such stochastic events cannot be predicted from instantaneous telemetry alone.

2. Subtle Tool Wear Combinations: Instances where cutter wear was elevated (>200 min) but spindle torque remained nominally low due to light cutting depth.

CHAPTER 11 – SYSTEM TESTING & VERIFICATION

11.1 Systematic Testing Suite

To verify software and predictive robustness, 8 comprehensive operational test cases were executed programmatically against the deployed model and pipeline. Table 11.1 documents the exact inputs, expected behaviors, actual model outputs, and pass/fail statuses.

Table 11.1: Comprehensive System Testing Suite (8 Mandatory Scenarios)

Test Case	Input Telemetry	Expected Behaviour	Actual Model Result	Status
TC-01: Normal Operation	Type: L, Air: 300K, Proc: 310K, RPM: 1500, Torque: 40Nm, Wear: 50m	Class 0 (No Failure, Low Risk)	No Machine Failure (1.10% risk)	**PASS**
TC-02: Higher Temperature	Type: L, Air: 304K, Proc: 313.5K, RPM: 1400, Torque: 45Nm, Wear: 80m	Diagnostic check (Thermal stress)	No Machine Failure (9.43% risk)	**PASS**
TC-03: Higher Torque Load	Type: L, Air: 300K, Proc: 310K, RPM: 1200, Torque: 70Nm, Wear: 120m	Class 1 (Machine Failure)	Machine Failure (95.00% risk)	**PASS**
TC-04: Higher Tool Wear	Type: L, Air: 301K, Proc: 311K, RPM: 1350, Torque: 62Nm, Wear: 240m	Class 1 (Tool Wear / Overstrain)	Machine Failure (100.00% risk)	**PASS**
TC-05: Machine Type H	Type: H, Air: 299K, Proc: 309K, RPM: 1520, Torque: 38Nm, Wear: 60m	Class 0 (Nominal for H-Type)	No Machine Failure (1.10% risk)	**PASS**
TC-06: Boundary Maximums	Type: M, Air: 304.5K, Proc: 313.8K, RPM: 1168, Torque: 76.6Nm,	Class 1 (Extreme Operational Limit)	Machine Failure (100.00% risk)	**PASS**

	Wear: 253m			
TC-07: Invalid Telemetry	Type: L, Air: 308K, Proc: 295K (Process < Air)	Validation warning raised in UI	Physical Inconsistency Warning Flagged	**PASS**
TC-08: Default Baseline	Type: L, Baseline parameters submitted	Class 0 (Safe baseline prediction)	No Machine Failure (1.10% risk)	**PASS**

CHAPTER 12 – LIMITATIONS AND FUTURE SCOPE

12.1 Project Limitations

1. Academic Educational Prototype: The system is designed for offline demonstration and viva defense, not for direct hardware PLC integration.
2. Aggregate Static Snapshots: Raw high-frequency acoustic emission signals were pre-condensed into aggregate statistical scalars.
3. Stochastic Failures: Purely random mechanical events (RNF mode) cannot be anticipated without continuous internal ultrasonic material testing.

12.2 Future Research & Technical Scope

1. IoT Edge Microcontroller Integration: Porting the scikit-learn pipeline to run on embedded Raspberry Pi 5 or ESP32 nodes communicating via industrial MQTT and OPC-UA.
2. Remaining Useful Life (RUL) Prognostics: Extending classification into survival regression models that forecast remaining cutting minutes before wear collapse.
3. Multi-Class Breakdown Classification: Predicting specific failure modes (e.g., differentiating between thermal dissipation failure and tool wear breakage) to guide automated maintenance inventory dispatch.

CHAPTER 13 – CONCLUSION

13.1 Concluding Remarks

This capstone project successfully engineered, evaluated, and deployed an end-to-end machine learning system for industrial machine failure prediction. Built strictly on legitimate open-source tooling and the genuine UCI AI4I 2020 dataset, the project demonstrated that foundational machine learning models can deliver reliable predictive maintenance diagnostics without commercial software overhead.

By implementing strict data leakage barriers and prioritizing Recall over misleading Accuracy, the champion Decision Tree classifier achieved an 83.82% detection rate on actual failures with only 11 False Negatives across 2,000 test observations. Serialized into an interactive Streamlit application and validated across 8 operational test scenarios, the project completely satisfies the official curriculum requirements of Learn Depth Academy LLP Track 1 Final Capstone – Problem 21.

REFERENCES

1. Matzka, S. (2020). AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5HS5C.

2. Mobley, R. K. (2002). An Introduction to Predictive Maintenance (2nd ed.). Elsevier Butterworth-Heinemann. ISBN: 978-0750675314.

3. Bousdekis, A., Lepenioti, K., Apostolou, D., & Mentzas, G. (2021). Decision making in predictive maintenance: literature review and research agenda for Industry 4.0. Computers & Industrial Engineering, 156, 107261.

4. Jardine, A. K., Lin, D., & Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. Mechanical Systems and Signal Processing, 20(7), 1483-1510.

5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

6. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

7. Streamlit Developers (2024). Streamlit Documentation: A faster way to build and share data apps. https://docs.streamlit.io/

APPENDICES

Appendix A – Important Source Code Summary

The implementation is structured into modular Python scripts:
• src/data_preprocessing.py: Ingestion, auditing, leakage prevention, and ColumnTransformer pipeline.
• src/train_model.py: Training Logistic Regression, KNN, and Decision Tree with balanced weighting.
• src/evaluate_model.py: Full metric computation, plot exports, and error analysis.
• src/run_tests.py: Verification test runner covering 8 operational scenarios.
• app.py: Streamlit reactive diagnostic web application.

Appendix B – Execution Commands (Windows CMD & PowerShell)

To reproduce the complete project on Windows:
1. Create virtual environment: py -3.12 -m venv .venv
2. Activate environment: .\.venv\Scripts\activate
3. Install dependencies: pip install -r requirements.txt
4. Execute training & evaluation: python src/evaluate_model.py
5. Execute test suite: python src/run_tests.py
6. Launch Streamlit web app: streamlit run app.py

Appendix C – Comprehensive Viva Questions and Answers (30+ Questions)

Q1: Why is this project significant for industrial manufacturing?

A1: Unplanned machinery failure causes millions of dollars in production downtime and poses grave safety hazards. Predictive maintenance detects early degradation before failure occurs.

Q2: What is the primary difference between preventive and predictive maintenance?

A2: Preventive maintenance adheres to rigid calendar intervals regardless of machine health. Predictive maintenance monitors real-time condition telemetry to service machinery only when degradation occurs.

Q3: Why is this problem formulated as binary classification?

A3: The operational goal is to output a clear decision: 0 (No Failure, continue operation) or 1 (Machine Failure imminent, intervene).

Q4: Which dataset was selected and what is its legitimate source?

A4: The AI4I 2020 Predictive Maintenance Dataset from the UCI Machine Learning Repository (DOI: 10.24432/C5HS5C, Stephan Matzka, 2020).

Q5: What are the primary features in the dataset?

A5: Machine Type (L, M, H), Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], and Tool wear [min].

Q6: How do you address the discrepancy between generic capstone inputs and dataset features?

A6: We strictly avoided fabricating fake columns. We explained the physical mapping: thermal sensors represent Temperature, torque and speed capture Vibration/Pressure forces, and tool wear tracks Runtime.

Q7: What is data leakage and how did your project prevent it?

A7: Data leakage occurs when training features contain information unavailable in production. We quarantined UDI and Product ID, and strictly excluded post-failure flags (TWF, HDF, PWF, OSF, RNF).

Q8: Why is ColumnTransformer fitted only on the training set?

A8: Fitting preprocessing on the complete dataset leaks test distribution statistics (mean, variance) into training, artificially inflating accuracy.

Q9: What is the target class balance in this dataset?

A9: Severe class imbalance: 9,661 nominal records (96.61%) and 339 failure records (3.39%), a 28.5:1 ratio.

Q10: Why is Accuracy an inappropriate metric for this problem?

A10: A naive model predicting 'No Failure' 100% of the time achieves 96.61% accuracy while catching zero breakdowns.

Q11: What is a False Negative in this industrial context?

A11: A False Negative occurs when the model predicts No Failure (0), but the machine breaks down (1). It is the most dangerous failure in predictive maintenance.

Q12: What is a False Positive and how does its cost compare to a False Negative?

A12: A False Positive is a false alarm where healthy machinery is flagged for inspection. It incurs minor inspection labor, which is vastly cheaper than catastrophic breakdown.

Q13: Why did you set random_state=42 throughout the code?

A13: It ensures deterministic, reproducible train-test splits and algorithmic weight initializations.

Q14: Why was stratified sampling used in train_test_split?

A14: Stratification guarantees that the 3.39% minority failure proportion is preserved equally across training and test partitions.

Q15: What foundational models were evaluated?

A15: Logistic Regression, K-Nearest Neighbors (KNN), and Decision Tree Classifier.

Q16: How did Logistic Regression perform on the test set?

A16: Accuracy = 82.45%, Recall = 82.35%, Precision = 14.18%, F1-score = 0.2419, FN = 12, FP = 339.

Q17: How did K-Nearest Neighbors perform on the test set?

A17: Accuracy = 97.35%, Precision = 82.61%, Recall = 27.94%, F1-score = 0.4176, FN = 49, FP = 4.

Q18: Why is KNN's 97.35% accuracy deceptive?

A18: KNN missed 49 out of 68 failures (72% missed!). It failed because majority class instances dominate localized neighborhoods.

Q19: How did the Decision Tree perform on the test set?

A19: Accuracy = 93.65%, Recall = 83.82%, Precision = 32.95%, F1-score = 0.4730, FN = 11, FP = 116.

Q20: Why was Decision Tree selected as the final champion model?

A20: It achieved the lowest False Negatives (FN = 11), highest Recall (83.82%), highest F1-score (0.4730), and provides clear physical rule splits.

Q21: What role did class_weight='balanced' play?

A21: It automatically penalized minority failure misclassifications inversely proportional to class frequency, forcing models to prioritize recall.

Q22: What does the correlation between Torque and Rotational Speed reveal?

A22: A strong negative correlation (-0.88), adhering directly to mechanical conservation of power (Power = Torque x Speed).

Q23: What did the error analysis reveal regarding missed failures?

A23: Missed failures predominantly corresponded to random failure (RNF) modes or subtle wear combinations with normal torque.

Q24: What is Streamlit and why was it chosen for deployment?

A24: Streamlit is an open-source Python framework that allows rapid deployment of reactive data science interfaces without frontend boilerplate.

Q25: Does the Streamlit application retrain the model on startup?

A25: No. It loads the pre-trained model and pipeline using joblib via cached resources for rapid inference.

Q26: What validation does the Streamlit application perform?

A26: It verifies input bounds and flags thermodynamic inconsistencies (such as Process Temp < Air Temp).

Q27: How many test cases were systematically verified?

A27: 8 distinct scenarios covering normal operation, thermal stress, torque overstrain, severe tool wear, machine variants, boundaries, and invalid inputs.

Q28: Why can this system not be considered a production deployment?

A28: It is an educational prototype tested on historical benchmark data, lacking direct industrial PLC communication and fail-safe hardware watchdogs.

Q29: How would you transition this prototype into production?

A29: By integrating IoT microcontrollers, streaming telemetry via OPC-UA/MQTT, and training on continuous time-series sensor streams.

Q30: What is the primary academic takeaway of this capstone?

A30: Careful data hygiene, strict leakage prevention, and metric-aligned model selection can produce high-performing industrial diagnostics using zero-cost foundational tools.